

PVW

Notizen: josia-heger.com/pvw

Ziel dieser Woche: Für 70 % der Lernziele die Theorie dazu verstehen,
und an leichten - mittleren Aufgaben anwenden können

Das Ziel nach dieser Woche ist nicht, dass ihr alles schon auswendig könnt, sondern dass ihr das meiste versteht und angewendet habt, damit ihr es nach dieser Woche mehrmals wiederholt, bis ihr es auswendig könnt.

Fragen:

- unbedingt viele Fragen stellen
- schreibt euch Fragen auf, wenn ihr am Nachmittag weiter lernt
- meldet euch falls es zu schnell oder zu langsam ist, oder wenn ihr für etwas nochmals eine genauere Erklärung wollt

Themen für heute: (Grundlagen), Zusammenhang, Kreise, (Matchings)

Pausen?

Potenz- und Logarithmusgesetze

$$a^n \cdot a^m = a^{m+n}$$

$$\underbrace{(a \cdot a \cdot a \cdot a)}_n \cdot \underbrace{(a \cdot a \cdot a)}_m = \underbrace{a \cdot a \cdot a \cdot a \cdot a \cdot a \cdot a}_n$$

$$\frac{a^n}{a^m} = a^{n-m}$$

$$\frac{\underbrace{(a \cdot a \cdot a \cdot a)}_n}{\underbrace{(a \cdot a)}_m} = \underbrace{a \cdot a}_{n-m}$$

$$a^n \cdot b^n = (a \cdot b)^n$$

$$\underbrace{(a \cdot a \cdot a \cdot a)}_n \cdot \underbrace{(b \cdot b \cdot b \cdot b)}_n = \underbrace{(a \cdot b) \cdot (a \cdot b) \cdot (a \cdot b) \cdot (a \cdot b)}_n$$

$$\frac{a^n}{b^n} = \left(\frac{a}{b}\right)^n$$

$$\frac{\underbrace{(a \cdot a \cdot a \cdot a)}_n}{\underbrace{(b \cdot b \cdot b \cdot b)}_n} = \underbrace{\left(\frac{a}{b}\right) \cdot \left(\frac{a}{b}\right) \cdot \left(\frac{a}{b}\right) \cdot \left(\frac{a}{b}\right)}_n$$

$$(a^n)^m = a^{n \cdot m}$$

$$\underbrace{\underbrace{(a \cdot a \cdot a)}_n \cdot \underbrace{(a \cdot a \cdot a)}_n \cdot \underbrace{(a \cdot a \cdot a)}_n \cdot \underbrace{(a \cdot a \cdot a)}_n}_{m} = \underbrace{a \cdot a \cdot a \cdot a \cdot a \cdot a \cdot a \cdot a \cdot a \cdot a \cdot a \cdot a}_{n \cdot m}$$

$$a^x = y \stackrel{\text{def}}{\Leftrightarrow} x = \log_a(y)$$

$$\log(x) + \log(y) = \log(x \cdot y)$$

$$\log(x^y) = y \cdot \log(x)$$

Notationen: $a^{-n} = \frac{1}{a^n}$

$a^n \rightarrow$ wir multiplizieren n mal mit a

$a^{-n} \rightarrow$ wir dividieren n mal mit a

$$a^{\frac{1}{n}} = \sqrt[n]{a}$$

$$\sqrt{a} \cdot \sqrt{a} = a, \quad a^{\frac{1}{2}} \cdot a^{\frac{1}{2}} = a^{\frac{1}{2} + \frac{1}{2}}$$

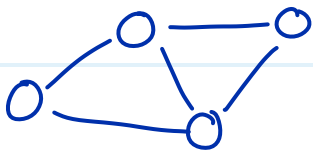
Zusammenhang

Definitionen:

G ist k -Knoten-zusammenhängend $\Leftrightarrow$ egal welche $k-1$ Knoten man entfernt, der Graph bleibt zusammenhängend

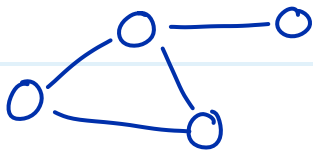
G ist k -Kanten-zusammenhängend $\Leftrightarrow$ egal welche $k-1$ Kanten man entfernt, der Graph bleibt zusammenhängend

Beispiele:



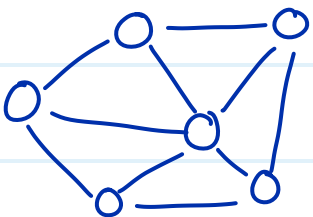
Knoten-zusammenhang = 2

Kanten-zusammenhang = 2



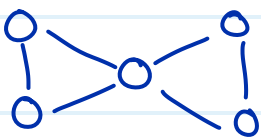
Knoten-zusammenhang = 1

Kanten-zusammenhang = 1



Knoten-zusammenhang = 3

Kanten-zusammenhang = 3



Knoten-zusammenhang = 1

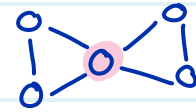
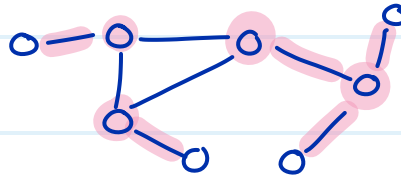
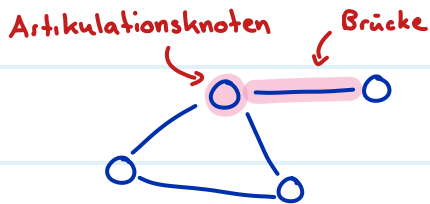
Kanten-zusammenhang = 2

Definition:

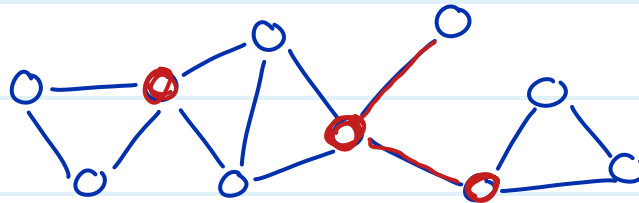
$e \in E$ ist eine Brücke $\Leftrightarrow G$ ist zusammenhängend aber $G - e$ ist nicht zusammenhängend

$v \in V$ ist ein Artikulationsknoten $\Leftrightarrow G$ ist zusammenhängend aber $G[V \setminus \{v\}]$ ist

nicht zusammenhängend

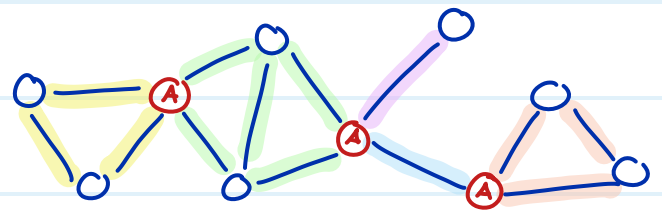


Beispiel:



Äquivalenzrelation:

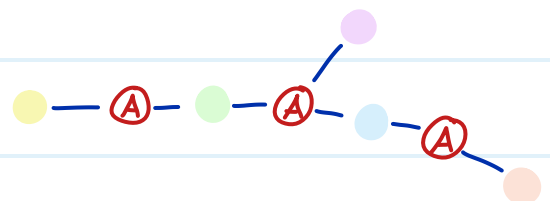
$e \sim f \stackrel{\text{def}}{\Leftrightarrow} e = f \text{ oder } e \text{ und } f \text{ liegen auf einem gemeinsamen Kreis}$



Blöcke = Äquivalenzklassen (= verschiedene Farben)

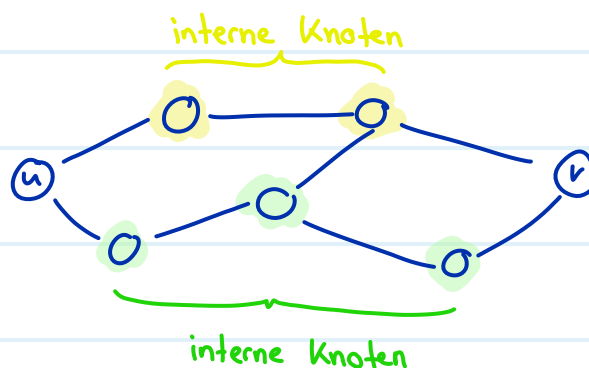
Blockgraph:

- ist immer ein Baum



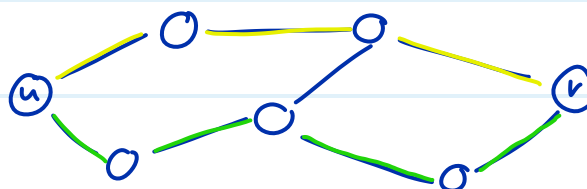
G ist k -Knoten-zusammenhängend $\Leftrightarrow$ egal welche $k-1$ Knoten man entfernt, der Graph bleibt zusammenhängend

Menger
 $\Leftrightarrow$ Für zwei verschiedene Knoten $u, v \in V$ gibt es mindestens k intern knotendisjunkte $u-v$ -Pfade

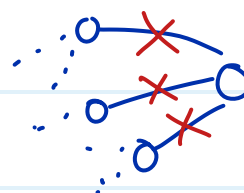


G ist k -Kanten-zusammenhängend $\Leftrightarrow$ egal welche $k-1$ Kanten man entfernt, der Graph bleibt zusammenhängend

Menger
 $\Leftrightarrow$ Für zwei verschiedene Knoten $u, v \in V$ gibt es mindestens k kantendisjunkte $u-v$ -Pfade



• Knotenzusammenhang $\leq$ Kantenzusammenhang $\leq$ minimaler Grad
 ↙ wegen Menger ↘



Tarjan Algorithmus: (ohne Namen in der Vorlesung)

- Findet Artikulationsknoten und Brücken

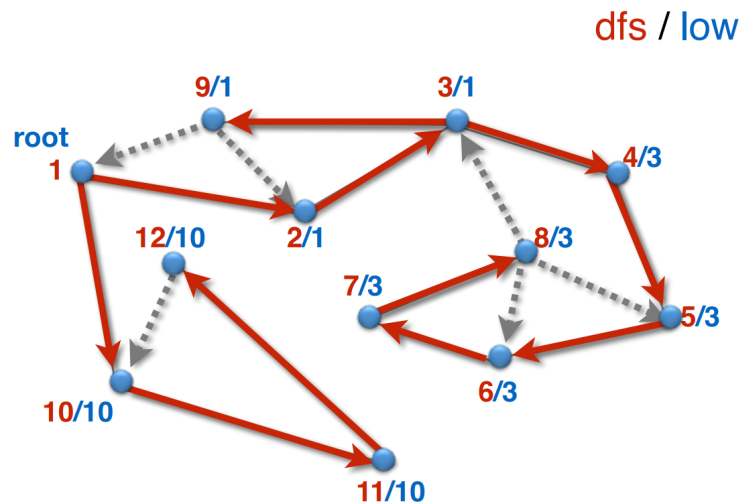
1) wir führen dfs aus von einem beliebigen Startknoten

$\text{dfs}[v] \stackrel{\text{def}}{=} \text{pre-Nummern}$

2) Baumkanten richten wir nach pre-Nummern aufsteigend (= vorwärts)

3) Restkanten richten wir nach pre-Nummern absteigend (= rückwärts)

4) $\text{low}[v] \stackrel{\text{def}}{=} \text{kleinste dfs Nummer, die wir von } v \text{ aus durch einen Pfad von beliebig vielen Baumkanten und max. einer Restkante erreichen können}$

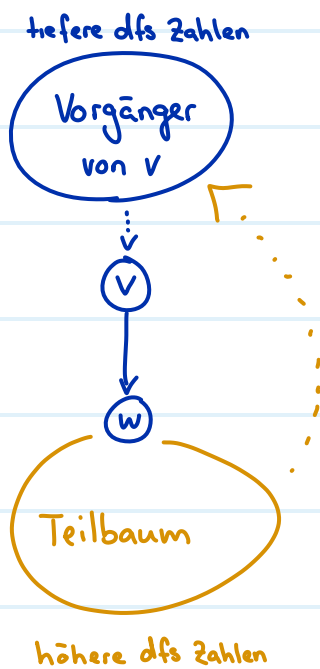


5) Startknoten s ist Artikulationsknoten $\Leftrightarrow s$ hat mindestens zwei Kinder
im dfs Baum

Erklärung: wenn wir s entfernen wären die beiden Kinder nicht mehr verbunden

$v \neq s$ ist ein Artikulationsknoten $\Leftrightarrow$ es gibt ein Kind w von v mit $\text{low}[w] \geq \text{dfs}[v]$

Erklärung:



es gibt ein Kind w von v mit $\text{low}[w] \geq \text{dfs}[v]$

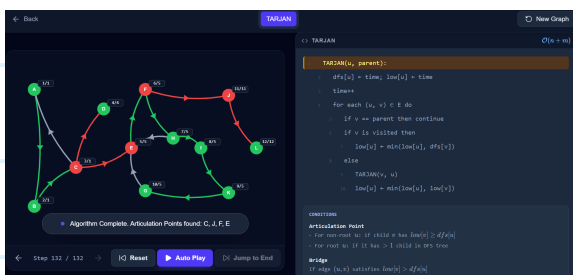
$\Leftrightarrow$ es gibt keine solche orange Restkante aus dem
 Teilbaum von w zu einem Vorgänger von v

$\Leftrightarrow$ Nach dem Entfernen von v ist ein Kind w von
 v mit keinem Vorgänger von v mehr verbunden

$\Leftrightarrow v \neq s$ ist ein Artikulationsknoten

6) $e = \{u, v\}$ ist eine Brücke $\Leftrightarrow (u, v) \in \text{dfs Baum}$ und $\text{low}[v] > \text{dfs}[u]$

Erklärung analog wie oben



Intuitives Verständnis viel wichtiger als genaue
 Implementierung. An der Prüfung kommen
 Theoriefragen dazu

Aufgaben

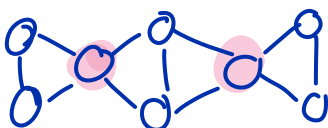
Die Funktion, die jedem Graphen seinen Blockgraphen zuweist, ist bijektiv.

- ☐ Wahr
☒ Falsch



Ein Graph kann zwei Artikulationsknoten und gleichzeitig keine Brücken haben.

- ☒ Wahr
☐ Falsch



G ist nicht 2-kanten-zusammenhängend $\implies G$ ist nicht 2-knoten-zusammenhängend.

- ☒ Wahr
☐ Falsch

$$A \rightarrow B \equiv \neg B \rightarrow \neg A$$

G hat drei Artikulationsknoten $\implies G$ ist 3-knoten-zusammenhängend.

- ☐ Wahr
☒ Falsch

Es gibt einen Algorithmus, der in $O(n + m)$ alle Artikulationsknoten und Brücken findet.

- ☒ Wahr
☐ Falsch

Sei s ein Artikulationsknoten in G . Wenn wir DFS bei s starten, dann wird s genau 1 Kind im DFS-Baum haben.

- ☐ Wahr
☒ Falsch

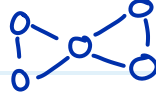
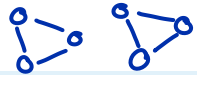
mindestens 2

Kreise

Eulertour: ein geschlossener Weg, der alle Kanten genau einmal enthält

Hamiltonkreis: ein Kreis, der alle Knoten genau einmal enthält

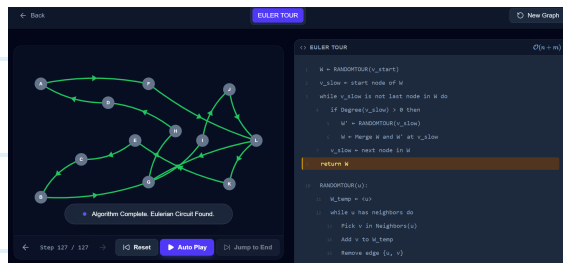
Richtig/falsch?

- Eine Eulertour ist ein Kreis $\rightarrow$ Falsch 
- Es gibt eine Eulertour $\Leftrightarrow \forall v \in V: \deg(v)$ ist gerade $\rightarrow$ Falsch 
- Es gibt eine Eulertour $\Leftrightarrow \forall v \in V: \deg(v)$ ist gerade und G ist zusammenhängend $\rightarrow$ Falsch: 

Wichtig: Es gibt eine Eulertour $\Leftrightarrow \forall v \in V: \deg(v)$ ist gerade und alle Kanten sind in derselben Zusammenhangskomponente

In jeden Knoten geht man so oft "hinein" wie "hinaus"

Algorithmus für Eulertour in $O(m)$:

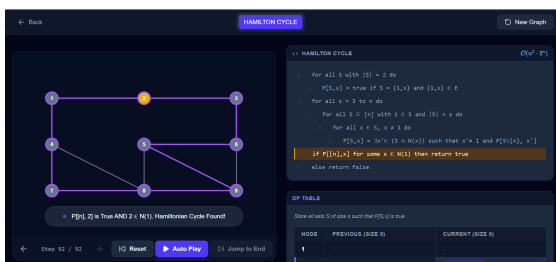


"greedy" Algorithm: Wir finden einen Kreis und entfernen ihn und fügen ihn in die bisherige Lösung ein

Intuitives Verständnis viel wichtiger als genaue Implementierung

Hamiltonkreis:

Teilproblem: $P_{S,x} = \begin{cases} 1, & \text{falls es einen 1-x-Pfad gibt, der genau die Knoten aus } S \text{ enthält} \\ 0, & \text{sonst} \end{cases}$



- NP-vollständig $\approx$ vermutlich nicht in polynomieller Zeit lösbar
- Anzahl Hamiltonkreise in $K_n = \frac{n!}{2n} \leftarrow$ alle möglichen Knotenreihenfolgen
 $\leftarrow$ jeder Kreis kann n Startpunkte haben und in zwei Richtungen gehen
 $= \frac{1}{2}(n-1)!$

Satz von Dirac: $\forall v \in V: \deg(v) \geq \frac{|V|}{2}$ und $|V| \geq 3 \Rightarrow G$ hat einen Hamiltonkreis

Beweis: Teil 1: Zusammenhang

Seien $x, y \in V$ beliebig, $x \neq y$

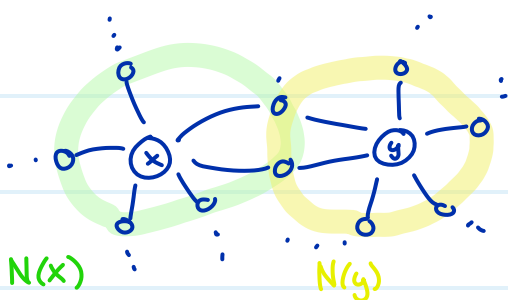
Case: $\{x, y\} \in E \Rightarrow$ es gibt einen x - y -Pfad

Case: $\{x, y\} \notin E$

$$\begin{aligned}
 \Rightarrow |N(x) \cap N(y)| &= |N(x)| + |N(y)| - |N(x) \cup N(y)| && \text{(Siebformel)} \\
 \underbrace{\text{Gemeinsame Nachbarn}} &\geq \frac{n}{2} + \frac{n}{2} - |N(x) \cup N(y)| && (\deg(x), \deg(y) \geq \frac{n}{2}) \\
 &\geq \frac{n}{2} + \frac{n}{2} - (n-2) && (x, y \notin N(x) \cup N(y)) \\
 &= 2
 \end{aligned}$$

$\Rightarrow$ Es gibt einen x - y -Pfad weil x und y einen gemeinsamen Nachbar haben

$\Rightarrow G$ ist zusammenhängend



$$n = 12$$

$$\deg(x), \deg(y) \geq 6$$

Teil 2: Hamiltonkreis

Sei $P = (v_1, v_2, \dots, v_k)$ ein längster Pfad in G

$\Rightarrow N(v_1) \subseteq P$ und $N(v_k) \subseteq P$ (sonst könnten wir P mit einem noch nicht verwendeten Knoten verlängern $\rightarrow$ Contradiction)

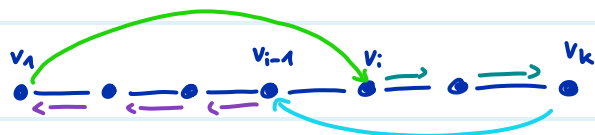
$\Rightarrow$ Es gibt zwei aufeinander folgende Knoten v_{i-1}, v_i

sodass $v_{i-1} \in N(v_k)$ und $v_i \in N(v_1)$

(Pigeonhole Prinzip:

$$\deg(v_1), \deg(v_k) \geq \frac{n}{2}$$

aber nur $n-1$ Möglichkeiten für v_i)



$\Rightarrow C = (v_1, v_i, v_{i+1}, \dots, v_{k-1}, v_k, v_{i-1}, v_{i-2}, \dots, v_2)$ ist ein Kreis mit k Knoten

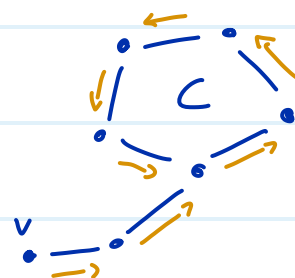
Case: Es gibt einen Knoten $v \in V$ der nicht in C ist

$\Rightarrow v$ ist verbunden mit dem Kreis C (Teil 1)

$\Rightarrow$ Wir schneiden C an der Verbindungsstelle auf und erstellen

so einen Pfad, der länger ist als P

$\Rightarrow$ Widerspruch, dieser Fall kann also nie eintreten

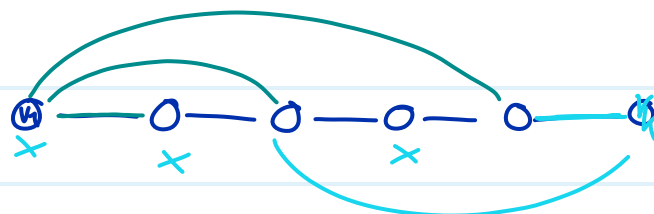


Case: Alle Knoten $v \in V$ sind in C enthalten

$\Rightarrow C$ ist per Definition ein Hamiltonkreis □

Dieser Beweis ist wichtig!

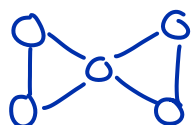
Bsp. $n = 6$



Aufgaben:

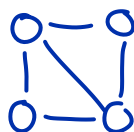
Es gibt einen Graphen, der eine Eulertour hat, aber kein Hamiltonkreis hat.

- ☒ Wahr
☐ Falsch



Es gibt einen Graphen, der einen Hamiltonkreis hat, aber keine Eulertour hat.

- ☒ Wahr
☐ Falsch

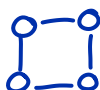


Es ist möglich in $O(m)$ festzustellen, ob es einen Hamiltonkreis gibt oder nicht.

- ☐ Wahr
☒ Falsch

Wenn $\forall v \in V : \deg(v) \leq \frac{|V|}{2}$ und $|V| \geq 3$ dann gibt es keinen Hamiltonkreis.

- ☐ Wahr
☒ Falsch



TSP

Gegeben: • Vollständiger Graph mit n Knoten

• Distanzen zwischen allen Knoten:

Abkürzung für $\{1, 2, \dots, n\} = V$

$\ell \cdot \binom{[n]}{2} \rightarrow \mathbb{R}$

Funktionsname ℓ für length

Jedem Knotenpaar wird eine Länge $\in \mathbb{R}$ zugeordnet

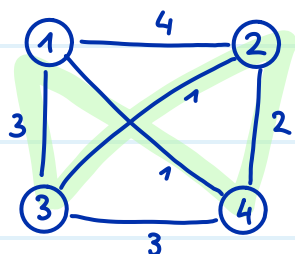
Menge aller Knotenpaare
= Kantenmenge = E

" $[n]$ choose 2"

Gesucht: minimale Länge eines Hamiltonkreises

(die Summe aller Kantengewichte $\sum \ell(e)$ vom Hamiltonkreis soll minimal sein)

Bsp.



$$V = [4] = \{1, 2, 3, 4\}$$

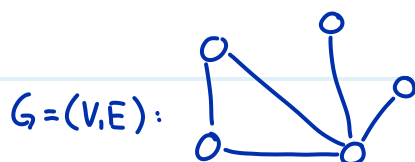
$$E = \binom{[4]}{2} = \left\{ \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\} \right\}$$

minimale Länge eines Hamiltonkreises? $\rightarrow 7$

NP vollständig: Wenn man TSP effizient lösen könnte, könnte man auch effizient bestimmen, ob es einen Hamiltonkreis gibt (... was sehr wahrscheinlich nicht geht)

Idee: wir formulieren das Hamiltonkreis-Problem zu einem TSP-Problem um

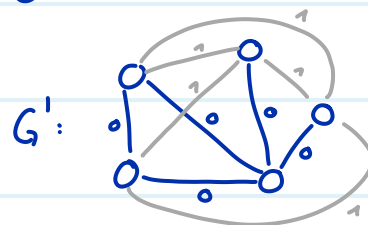
$$\exists \text{ Hamiltonkreis in } G \iff \text{TSP Lösung von } G' = 0$$



Konstruktion von G'

fehlende Kanten $\rightarrow \ell(e) := 1$

Kanten in $E \rightarrow \ell(e) := 0$

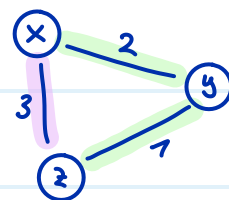


Deshalb kann es auch keinen C -Approximationsalgorithmus geben für $C > 0$

→ Sehr hoffnungslos...

Metrisches TSP: erfüllt zusätzlich die Dreiecksungleichung

$$\forall x, y, z \in V: \ell(x, z) \leq \ell(x, y) + \ell(y, z)$$



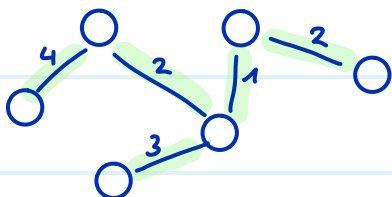
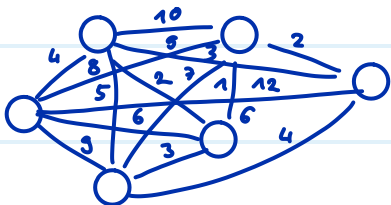
$\Leftrightarrow$ Abkürzen macht die Länge eines Weges kürzer oder sie bleibt gleich

Aufgabe: Sei $P = (a, b, c, d)$ ein Pfad. Zeige dass $\ell(a, d) \leq \ell(P)$ mit der Annahme, dass die Dreiecksungleichung gilt.

$$\begin{aligned} \ell(P) &= \ell(a, b) + \ell(b, c) + \ell(c, d) && \searrow \text{Dreiecksungleichung mit } a, b, c \\ &\geq \ell(a, c) + \ell(c, d) && \searrow \text{Dreiecksungleichung mit } a, c, d \\ &\geq \ell(a, d) \end{aligned}$$

2-Approximationsalgorithmus für METRISCHES TSP (nicht für normales TSP)

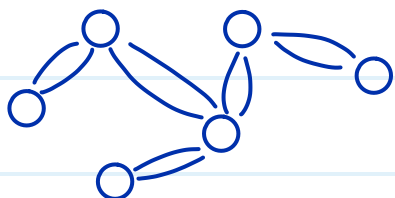
sei OPT die optimale Lösung



1. Finde MST

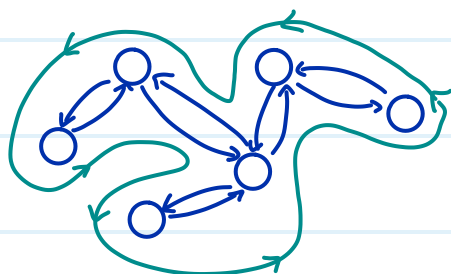
$$\ell(\text{MST}) \leq \text{OPT}$$

↳ teuerste Kante von OPT entfernen ergibt einen Spannbaum



2. Verdopple MST Kanten

$$2 \cdot \ell(\text{MST}) \leq 2 \cdot \text{OPT}$$

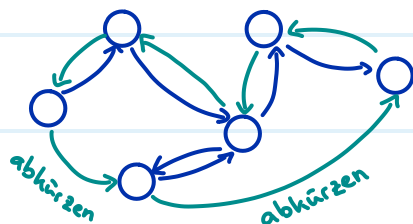


3. $W :=$ Eulertour der verdoppelten MST Kanten
einfach einmal "um den MST herum laufen"

$$\ell(W) = 2 \cdot \ell(\text{MST}) \leq 2 \cdot \text{OPT}$$

↳ def Eulertour

4. $C :=$ Laufe W ab und kürze ab
(bereits verwendete Knoten überspringen)



$$\ell(C) \leq \ell(W) \leq 2 \cdot \text{OPT}$$

↳ abkürzen, Dreiecksungleichung

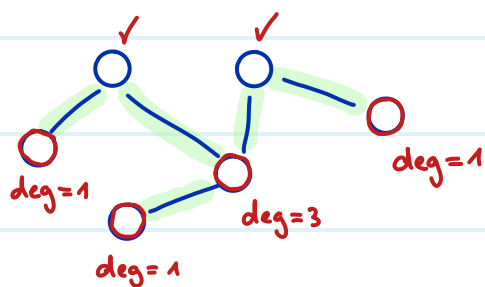
- C ist ein Hamiltonkreis weil wir jeden Knoten genau einmal besuchen,
- Approximation ist im worst case doppelt so lange wie OPT
- Laufzeit $O(n^2)$ mit Adjazenzmatrix

1.5-Approximationsalgorithmus für METRISCHES TSP (nicht für normales TSP)

Frage: Wieso haben wir die MST Kanten verdoppelt?

→ Damit alle Knoten einen geraden Grad haben bezüglich der verdoppelten Kanten, und eine Eulertour existiert

Idee Statt alle Kanten zu verdoppeln, fügen wir nur Kanten zwischen den "Problemknoten" ein, die einen ungeraden Grad haben im MST



Anzahl Knoten mit ungeradem Grad ist gerade
(Handschlaglemma)

Sei M ein gewichtsminimales perfektes Matching der Problemknoten

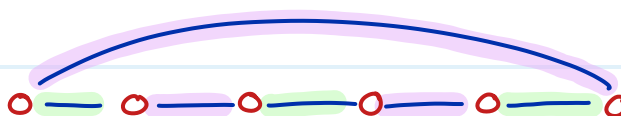
$$\ell(M) \leq \frac{1}{2} \text{OPT}$$

Beweis: Sei OPT_p die Länge der optimalen TSP Route, wenn wir nur die Problemknoten betrachten

$$\text{OPT}_p \leq \text{OPT}$$

↳ abkürzen

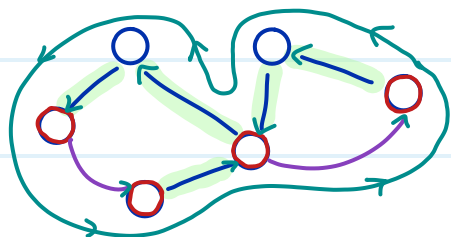
wir teilen die Kanten abwechselnd in zwei Mengen (A und B)



$$\text{entweder } \ell(A) \leq \frac{1}{2} \text{OPT}_p \text{ oder } \ell(B) \leq \frac{1}{2} \text{OPT}_p$$

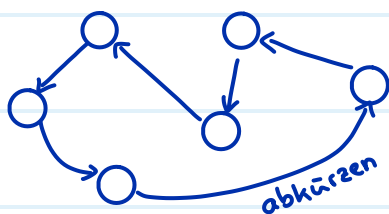
$$\text{weil } \ell(A) + \ell(B) = \text{OPT}_p$$

die kleinere Menge A oder B ist ein Matching mit Gewicht $\leq \frac{1}{2} \text{OPT}$



3. $W = \text{Eulertour mit den Kanten von MST u } M$

$$\ell(W) = \ell(\text{MST}) + \ell(M) \leq \text{OPT} + \frac{1}{2} \text{OPT} = 1.5 \text{OPT}$$



4. abkürzen (wie vorher)

$$\ell(C) \leq \ell(W) \leq 1.5 \text{OPT}$$

$\Rightarrow$ das ist ein 1.5-Approximationsalgorithmus



Es gibt einen 2-Approximationsalgorithmus für das TSP, der Laufzeit $O(n^2)$ hat.

☐ Wahr

☒ Falsch

„metrisch“ bedeutet, dass wenn wir einen Knoten überspringen und so einen Pfad abkürzen, die Länge des Pfades dann gleich bleibt oder kleiner wird.

☒ Wahr

☐ Falsch

Es gibt einen 1.5-Approximationsalgorithmus in Laufzeit $O(n^3)$ für das metrische TSP.

☒ Wahr

☐ Falsch

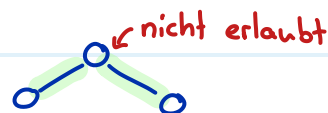
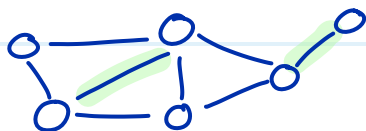
Falls wir das TSP in polynomieller Laufzeit lösen könnten, dann könnten wir auch in polynomieller Zeit bestimmen, ob ein Graph einen Hamiltonkreis hat.

☒ Wahr

☐ Falsch

Matchings

- Ein Matching M ist eine Menge von Kanten $M \subseteq E$, sodass kein Knoten zu mehr als einer Matchingkante inzident ist



- Ein Knoten v wird von M **überdeckt**, wenn v zu einer Matchingkante inzident ist
- M ist **perfekt** $\Leftrightarrow$ alle Knoten v werden von M überdeckt

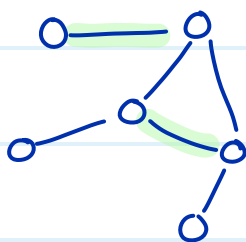
$$\Leftrightarrow |M| = \frac{|V|}{2}$$

- M ist **kardinalitätsmaximal** $\Leftrightarrow |M| \geq |M'|$ für alle Matchings M'

$\Leftrightarrow$ Es gibt kein Matching mit mehr Kanten

- M ist **inklusionsmaximal** $\Leftrightarrow M \cup \{e\}$ ist kein Matching für alle $e \in E \setminus M$

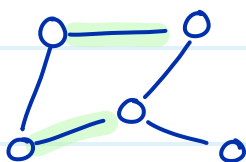
$\Leftrightarrow$ Es gibt kein Matching M' mit $M \subseteq M'$ und $|M| < |M'|$



perfekt? $\rightarrow$ Nein

kardinalitätsmaximal? $\rightarrow$ Nein

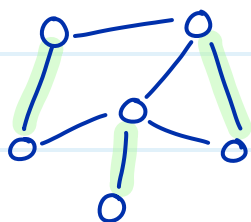
inklusionsmaximal? $\rightarrow$ Ja



perfekt? $\rightarrow$ Nein

kardinalitätsmaximal? $\rightarrow$ Ja

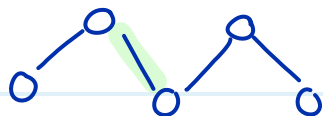
inklusionsmaximal? $\rightarrow$ Ja



perfekt? $\rightarrow$ Ja

kardinalitätsmaximal? $\rightarrow$ Ja

inklusionsmaximal? $\rightarrow$ Ja



perfekt? $\rightarrow$ Nein

kardinalitätsmaximal? $\rightarrow$ Nein

inklusionsmaximal? $\rightarrow$ Nein

Wie "gut" ist ein Matching?

$\mathcal{P}(E)$ = Menge aller Teilmengen von E

Matchings

inklusionsmaximal

kardinalitätsmaximal

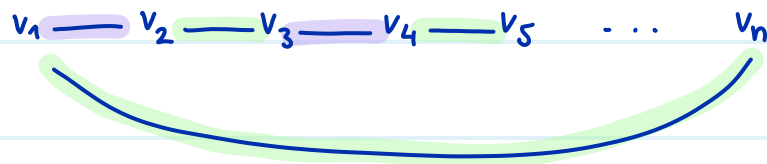
perfekt

Let $G = (A \cup B, E)$ be a bipartite graph.

$$M_1 \cap M_2 = \emptyset$$

- (a) **True or False:** If G has a Hamiltonian cycle, then G contains two disjoint perfect matchings.

Number of edges in Hamiltoncycle is even (because G is bipartite)



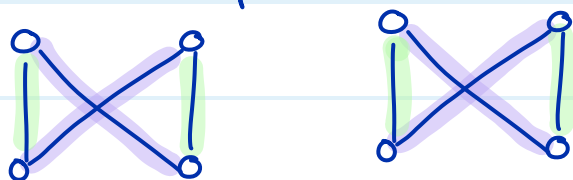
M_1 = even edges

M_2 = odd edges

$\Rightarrow$ True

- (b) Prove or disprove: If G contains two disjoint perfect matchings, then G has a Hamiltonian cycle.

Counterexample:



1. Let $G = (V, E)$ be a graph with a perfect matching.

- **Prove or Disprove:** G must be a connected graph.

Counterexample:



3. Let G be a graph with an odd number of vertices.

- **Prove or Disprove:** G cannot have a perfect matching.

Proof by contradiction. Assume M is a perfect matching in G

$$\Rightarrow |M| = \frac{|V|}{2} \quad \text{by definition}$$

$\Rightarrow |V|$ must be even, contradiction

5. Let T be a tree with more than 1 vertex.

- **Prove or Disprove:** T must have a perfect matching.

Counterexample:

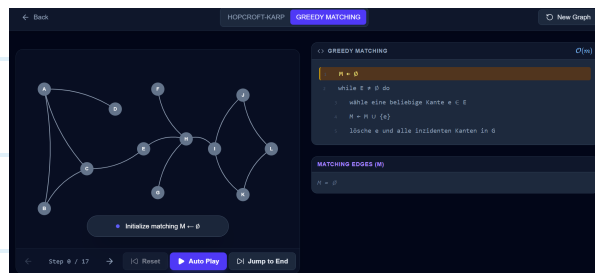


Greedy Algorithm:

GREEDY-MATCHING (G)

in $O(|E|)$

- 1: $M \leftarrow \emptyset$
- 2: **while** $E \neq \emptyset$ **do**
- 3: wähle eine beliebige Kante $e \in E$
- 4: $M \leftarrow M \cup \{e\}$
- 5: lösche e und alle inzidenten Kanten in G

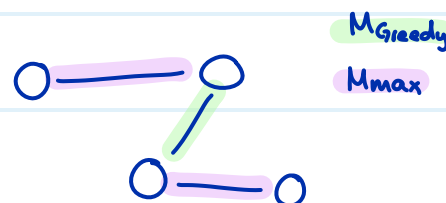


• M_{Greedy} ist inklusionsmaximal

• $|M_{\text{Greedy}}| \geq \frac{1}{2} |\text{kardinalitätsmaximales Matching } M_{\text{max}}|$

mindestens ein Endpunkt jeder Kante aus M_{max} muss von M_{Greedy}

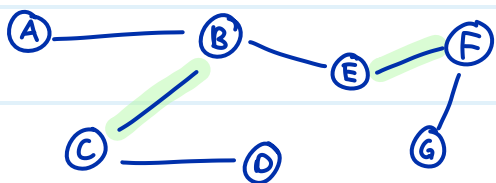
überdeckt werden (sonst könnten wir sie zu M_{Greedy} hinzufügen $\rightarrow$ Contradiction)



Frage: Gegeben ein Matching, wie könnte man es verbessern?

Idee: Ein M -augmentierender Pfad ist ein Pfad der abwechselnd Kanten aus

M und nicht aus M verwendet, und mit unüberdeckten Knoten beginnt und endet.



Augmentierende Pfade. $P_1 = (A, B, C, D)$

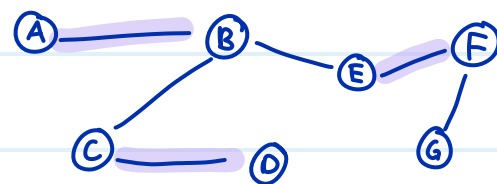
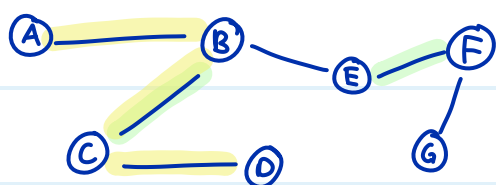
$P_2 = (D, C, B, E, F, G)$

$M = \{\{B, C\}, \{E, F\}\}$

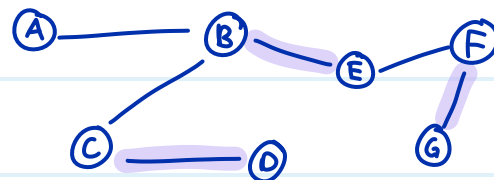
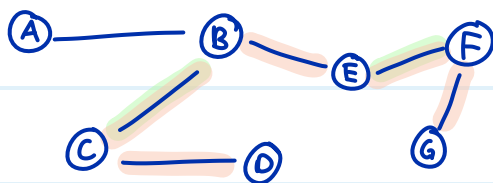
$$M \oplus P_1$$

$\oplus \hat{=}$ XOR

$\hat{=}$ addition modulo 2



$$M \oplus P_2$$



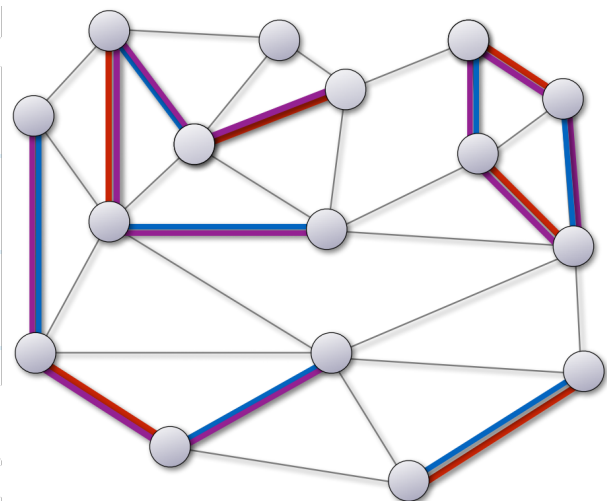
Durch Augmentieren wächst $|M|$ um 1

Achtung: P_2 ist kein augmentierender Pfad für $(M \oplus P_1)$

Satz von Berge Jedes nicht kardinalitätsmaximale Matching hat einen augmentierenden Pfad

Beweis: Sei M ein nicht kardinalitätsmaximales Matching

und $M_{\max}$ ein kardinalitätsmaximales Matching



Wir betrachten $M \oplus M_{\max}$

$\Rightarrow$ Jeder Knoten hat $\text{Grad} \leq 2$
(höchstens +1 von M und +1 von $M_{\max}$)

$\Rightarrow$ Graph ist in Pfade und Kreise zerlegbar

$\Rightarrow$ Alle diese Pfade und Kreise wechseln zwischen Kanten aus M und aus $M_{\max}$.

$\Rightarrow$ Kreise haben gerade Längen und benutzen deshalb gleich viele Kanten aus M wie aus $M_{\max}$

$\Rightarrow$ Weil $|M| < |M_{\max}|$ muss es mindestens einen Pfad P ungerader Länge geben,
der mehr Kanten aus $M_{\max}$ als aus M verwendet

$\Rightarrow$ Die Endpunkte von P werden von $M_{\max}$ überdeckt aber nicht von M

$\Rightarrow P$ ist ein augmentierender Pfad

